

Scalable Stream Processing Using Apache Pulsar

Tejas Chakravarthy, Andreas Hadjoulis, Julios Fotiou, Tanay Singh, Adnan Erlangga Raharaja
Uppsala University

1TD076 Data Engineering II, VT 2026, Group 16

Source code: <https://github.com/tj-chakravarthy/Data-Engineering-II-Project-2>

Abstract—This paper presents a scalable GitHub analytics system using Apache Pulsar. The system collects repository metadata from the GitHub API, enriches it using additional API requests, and continuously aggregates the results to answer four analytics questions about languages, commits, unit tests, and CI/CD usage. The system is deployed with Docker and Docker Swarm across cloud virtual machines. A 38-hour collection processed 78,110 unique repositories. The experiments show that runner parallelism and GitHub API token count have the strongest effect on throughput, while worker VM count and batch size have smaller effects.

Index Terms—GitHub API, Apache Pulsar, stream processing, containerization, orchestration, Docker Swarm, data analytics

I. INTRODUCTION

GitHub hosts a large number of open-source software projects and provides repository metadata through its REST API [1]. This metadata can be useful for understanding software development trends, such as which programming languages are most common, which repositories are actively maintained, and how often projects use unit tests and continuous integration. However, these higher-level analytics are not directly available through GitHub search. Therefore, a data engineering pipeline is needed to collect, process, and analyze repository metadata at scale.

This project develops a GitHub analytics system using Apache Pulsar as the streaming framework [2]. The system collects repository metadata from the GitHub API, publishes it to Pulsar topics, and processes it using worker services to produce the required analytics. The analytics component is designed to answer the project questions, including the most common programming languages, the most frequently updated repositories, the languages most associated with unit testing, and the languages most associated with both unit testing and continuous integration. The system also addresses practical data engineering challenges. GitHub search results are limited, so the crawler must divide the collection process into smaller queries and use pagination. API rate limits must also be considered when collecting repository metadata. To make the system reproducible and easier to deploy, the implementation uses Docker containers and automation scripts. The main contribution of this work is the design of a cloud-based and scalable GitHub analytics pipeline using suitable data engineering frameworks. Apache Pulsar is used as the streaming layer, while containerization and automation support deployment across multiple virtual machines.

II. RELATED WORK

GitHub is widely used as a data source in empirical software engineering and mining software repository research. Kalliamvakou et al. show that GitHub metadata can provide useful information about software development activity, but also emphasize that such data must be interpreted carefully [3]. This is relevant to the present project because repository metadata is used to derive analytics about programming languages, update activity, unit testing, and continuous integration.

Stream processing systems are useful when data should be processed incrementally rather than only after a complete dataset has been collected. Blamey et al. compare stream-processing systems using throughput-oriented benchmarks under different processing loads [4]. In this project, Apache Pulsar is used as the streaming layer, following a producer-topic-consumer model where repository records are published as messages and processed by worker services [2].

Cloud deployment and automation are also relevant because the system is intended to run across multiple virtual machines. Armstrong et al. discuss cloud contextualization as a way to configure virtual machines during deployment [5]. In this project, containerization and automation are used to make the pipeline more reproducible and easier to deploy across cloud resources.

III. SYSTEM ARCHITECTURE

A. Overview

The proposed system is designed as a cloud-based streaming pipeline for collecting, enriching, and analyzing GitHub repository data. Figure 1 shows the high-level architecture of the system. The architecture separates the pipeline into three main stages: data collection, repository metadata enrichment, and data aggregation. This separation allows the runners responsible for enriching repository metadata to be developed, deployed, and scaled independently.

The pipeline starts with a crawler that retrieves repository metadata from the GitHub API. Each repository is converted into a structured record and published to a raw repository stream. A configurable number of processing runners consume these records and enrich them with additional information using the GitHub API. This enrichment is necessary because the initial metadata is not sufficient to answer all project questions, especially those related to commit activity, unit-test usage, and CI/CD usage. The runners then publish data to an aggregation stream. Finally, the aggregator computes a new answer to each question based on all aggregated data so far.

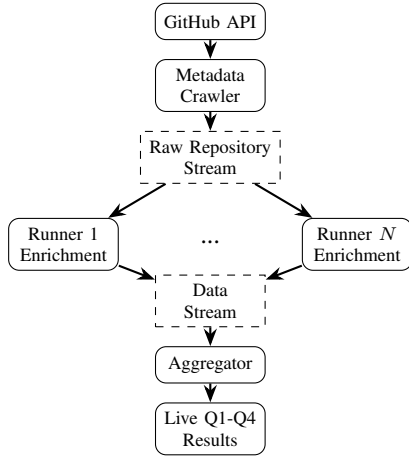


Fig. 1: High-level architecture of the GitHub analytics system. The crawler collects repository metadata, runners enrich the records using additional GitHub API requests, and the aggregator continuously updates the analytics results.

B. Deployment and Automation

The system is intended to run across multiple cloud virtual machines using containerized services. The master node will host the coordination services, while the worker nodes will run the services responsible for data processing. Docker is used to containerize each service, while Docker Swarm is used for orchestrating these services across a cluster of one manager and up to four workers. This deployment model supports scalability experiments by allowing the number of runners and the number of worker machines to be changed.

C. Stream Processing with Apache Pulsar

Apache Pulsar is used as the streaming layer between the crawler, runners, and aggregator. The crawler acts as a producer, while the runners act as both consumers and producers. Finally, the aggregator consumes from the runners. As a result, the crawler does not need to know how many runners are active, and the processing stage can be scaled independently. The streaming layer also helps improve locality by distributing records to runners on worker nodes instead of using a single centralized enrichment component. Each runner processes its assigned records locally and publishes the data back to the stream. This reduces unnecessary data movement while allowing more runners or worker machines to be added without changing the crawler or aggregator.

D. Data Collection

The crawler is responsible for collecting repository metadata from the GitHub API. Since the GitHub search interface limits the number of results returned by a single query, the crawler divides the collection process into smaller queries and uses pagination. Crawling can take a significant amount of time, particularly for large datasets, and the GitHub API also imposes request limits. Re-crawling data for days that have already been collected would therefore be an inefficient use

of resources. For this reason, collected results are cached on disk for each day and reused across runs whenever data for that day is needed again. The crawler also removes duplicate repositories before publishing records to the stream. This ensures that each repository is processed only once by the downstream stages.

The collected metadata provides the initial information needed by the pipeline. Some analytics can be derived directly from this metadata, while others require additional enrichment through the GitHub API. Therefore, the crawler only performs the initial collection step, and the runners are responsible for gathering any extra information needed by the later analytics stage.

E. Enrichment and Aggregation

The runners enrich repository records by retrieving the additional information needed for the analytics questions. Commit activity is estimated by querying the GitHub commits endpoint, while unit-test and CI/CD indicators are detected by checking selected repository paths and configuration files. This avoids downloading the full repository while still providing the extra information needed for the analytics stage.

To reduce communication overhead, runners can send data in batches of configurable size. The aggregator consumes these batches and re-evaluates the current global answers to the four project questions. Since aggregation is performed continuously, the system produces live results that are updated as more repositories are processed.

The analytics stage also supports adaptive top- N queries. The runners extract general repository features, while the aggregator computes the final rankings. The value of N is provided as a runtime configuration parameter, through the deployment environment file. Therefore, changing from top-10 to top-20 only requires updating the configuration and restarting the aggregator component, without changing the crawler, runners, or streaming layer.

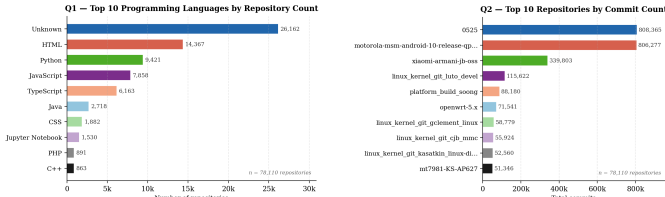
IV. EXPERIMENTS AND RESULTS

A. GitHub Analytics Results

The system collected data over 38 hours, during which 78,110 unique repositories were processed and used to answer the four project questions. Of these, 36,185 were served from the crawler cache of a previous run, with nearly half the dataset reused rather than re-crawled.

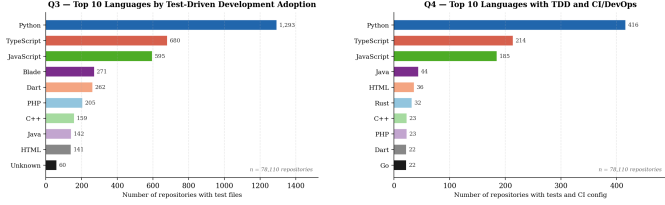
1) *Q1: Top Languages by Repository Count*: Figure 2a shows the top 10 languages by repository count. The largest group is Unknown (26,162), roughly a third of repositories with no detected primary language. Among named languages, HTML (14,367), Python (9,421), JavaScript (7,858), and TypeScript (6,163) lead, reflecting the prominence of web and general-purpose languages in the collected dataset.

2) *Q2: Most Frequently Updated Repositories*: Figure 2b shows the top 10 repositories by commit count. The leading entries are mainly kernel forks and mirrors, with the top two exceeding 800,000 commits. This shows that raw commit



(a) Top 10 programming languages by repository count. (b) Top 10 repositories by total commit count.

Fig. 2: Repository language and commit count distributions ($n = 78,110$).



(a) Top 10 languages by TDD adoption. (b) Top 10 languages by combined TDD and CI/DevOps adoption.

Fig. 3: TDD and combined TDD-CI/DevOps adoption across programming languages ($n = 78,110$).

count must be interpreted carefully, since mirrored repositories can appear disproportionately active.

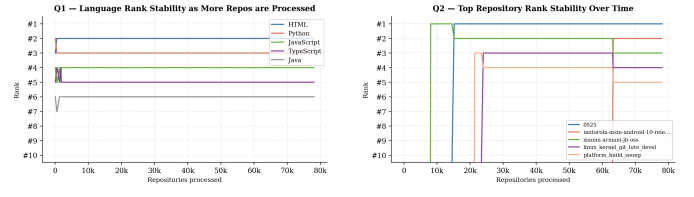
3) *Q3: Languages with Test-Driven Development*: Figure 3a shows the top 10 languages by repositories containing test files. Python leads with 1,293 repositories, followed by TypeScript (680) and JavaScript (595). Overall, the results suggest that test adoption is relatively low compared with the total number of processed repositories.

4) *Q4: Languages with TDD and CI/DevOps*: Figure 3b shows languages with both test files and CI/CD configuration. Python (416), TypeScript (214), and JavaScript (185) lead again. However, only a fraction of the repositories with test indicators also contain CI/CD configuration, showing that testing and CI/CD adoption do not always appear together.

5) *Live Results Stability*: Figures 4 and 5 show that the rankings stabilize early during processing. Q1 stabilizes almost immediately, with the leading languages keeping fixed positions for the rest of the dataset. Q2 also converges early among the top repositories, although some lower-ranked entries shift later. For Q3 and Q4, Python, TypeScript, and JavaScript remain the dominant languages after brief initial fluctuations.

B. Experiments

1) *Experimental Setup*: The experiments evaluate the scalability and performance of the GitHub analytics pipeline under different deployment configurations. Each run was executed for 30 minutes. The main metrics were the total number of processed repositories, throughput in repositories per minute, and average end-to-end latency in seconds. Latency was measured as the time between a repository being emitted by the



(a) Q1 language ranks stabilize within the first 1,000 repositories. (b) Q2 repository ranks stabilize within the first 1,000 repositories.

Fig. 4: Rank-stability trends for Q1 (languages) and Q2 (repositories).

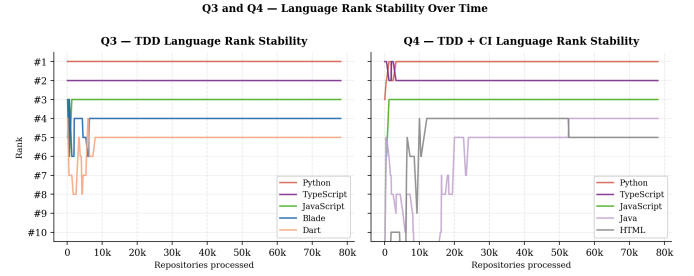


Fig. 5: Language-rank stability over time for Q3 (left) and Q4 (right).

TABLE I: Scalability and batching experiments across runners, GitHub API tokens, worker VMs, and batch sizes.

Experiment	Scale	Repos	Throughput	Latency
Runner	1 runner	700	23.33	871.94
Runner	2 runners	1520	50.67	561.43
Runner	4 runners	1960	65.33	291.74
Runner	8 runners	1920	64.00	135.29
GitHub API token	1 token	380	12.67	136.31
GitHub API token	3 tokens	1200	40.00	261.60
GitHub API token	5 tokens	1980	66.00	279.41
Worker VM	1 worker	1980	66.00	323.93
Worker VM	2 workers	2000	66.67	338.28
Worker VM	4 workers	1980	66.00	310.87
Batch size	1	2002	66.73	281.83
Batch size	20	1960	65.33	297.46
Batch size	100	1800	60.00	333.27

crawler and the corresponding result being received by the aggregator.

The evaluation followed a staged design instead of a full grid search. Each experiment group varied one parameter while keeping the others fixed. The tested parameters were the number of analytics runners, the number of GitHub API tokens, the number of worker VMs, and the batch size used when runners send results to the aggregator.

2) *Runner Scalability*: The runner scalability experiment varied the number of runners from 1 to 8 while keeping the number of workers, tokens, and batch size fixed. The results are shown in Table I and Figures 6, 7, and 8.

Increasing the number of runners had the strongest effect on performance. Throughput increased from 23.33 repositories/minute with one runner to about 65 repositories/minute

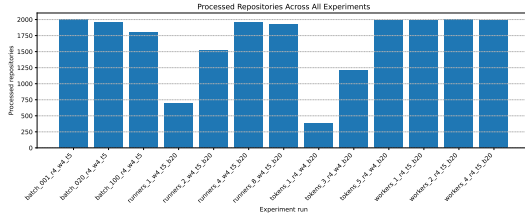


Fig. 6: Processed repositories across all experiment runs.

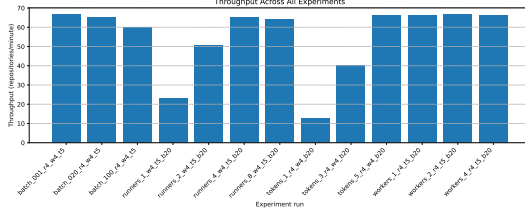


Fig. 7: Throughput across all experiment runs.

with four runners. Increasing the number of runners to eight did not improve throughput further, suggesting that the system reaches a bottleneck after four runners. This is likely caused by GitHub API access and the limited number of available tokens. However, latency continued to decrease with more runners, which suggests that additional runners reduce the time that repositories spend waiting before being processed, even when total throughput is already capped.

3) *Token Scaling*: The token scaling experiment varied the number of GitHub API tokens while keeping four runners, four workers, and batch size 20. The results are shown in Table I and Figures 6, 7, and 8.

Token count also clearly impacted throughput. With one token, the system processed only 380 repositories, while with five tokens it processed 1980 repositories. Throughput increased from 12.67 to 66.00 repositories/min. This confirms that API access is a major limiting factor. The latency results are less direct: latency increases as more tokens are added. This does not necessarily mean that the system becomes slower per repository. Rather, more tokens allow many more repositories to enter and stay active in the pipeline, which can increase the measured end-to-end time for completed records.

4) *Worker VM Scalability*: The scaling experiment varied the number of worker VMs while keeping 4 runners, 5 tokens, and batch size 20. Results are shown in Table I and Figures 6, 7, and 8.

Increasing the number of worker VMs had almost no effect on throughput. Throughput stayed close to 66 repositories/minute in all three configurations. This suggests that the workload is not computationally expensive enough to benefit significantly from more machines. Most time is likely spent waiting for GitHub API requests rather than performing CPU-intensive local processing. Therefore, worker VM scaling is less important than runner parallelism and token availability for this workload.

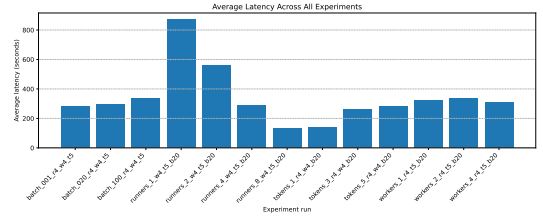


Fig. 8: Average end-to-end latency across all experiment runs.

5) *Batch Size*: The batch-size experiment varied how often runners send results to the aggregator. The results are shown in Table I and Figures 6, 7, and 8.

Batch size had only a small effect compared with runners and tokens. Smaller batches gave slightly higher throughput and lower latency, but the difference was not large enough to identify batching as the main bottleneck. This suggests that communication through Pulsar and aggregation are not as expensive as the external GitHub API enrichment work. Larger batches may delay updates because records wait longer before being sent to the aggregator, which can explain the higher latency for batch size 100.

C. Discussion

Overall, no experiment run processed more than 2000 repositories within the 30-minute window. During the enrichment phase, each repository can require between 3 and 19 extra calls to the GitHub API. Reaching roughly 2000 repositories therefore coincides with the point at which all GitHub tokens are drained and no further work can be done.

V. CONCLUSION

This project built a scalable stream-processing system for GitHub analytics. The pipeline processed 78,110 unique repositories and answered all four analytics questions. A method for automatically assigning floating IP addresses to VMs via the OpenStack API was also developed and added to the project repository, but was not included in the final submission. Experiments showed that runners and GitHub API tokens affected throughput the most, while worker VMs and batch size had limited impact. Overall, performance was mainly constrained by GitHub API access.

REFERENCES

- [1] GitHub, “Github rest api documentation,” <https://docs.github.com/en/rest>, 2026, accessed: 2026-05-21.
- [2] Apache Software Foundation, “Apache pulsar documentation,” <https://pulsar.apache.org/docs/>, 2026, accessed: 2026-05-21.
- [3] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “The promises and perils of mining github,” in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 92–101.
- [4] B. Blamey, A. Hellander, and S. Toor, “Apache spark streaming, kafka and harmonico: a performance benchmark and architecture comparison for enterprise and scientific computing,” in *International Symposium on Benchmarking, Measuring and Optimization*. Springer, 2019, pp. 335–347.
- [5] D. Armstrong, D. Espling, J. Tordsson, K. Djemame, and E. Elmroth, “Contextualization: dynamic configuration of virtual machines,” *Journal of Cloud Computing*, vol. 4, no. 1, p. 17, 2015.